nomad-ai-factory 코드 리뷰

리뷰 일자: 2026-10-11 / 대상 경로: D:₩WorkSpace₩platform₩nomad-ai-factory (v1) / 비교 참조: D:₩WorkSpace₩platform₩Nomad-AI-Factory_V.2 (기획 문서만 존재하는 차기 버전)

1. 프로젝트 개요

- 한 줄 설명: 미국 구매대행(Amazon/eBay) + 국내 위탁판매 소싱 -> AI 상품 리스팅/블로그/SNS 카피 생성 -> 자동화 캔버스까지 묶은 "글로벌 셀러 올인원 플랫폼" 프로토타입. Vite SPA 프론트 + Express 프록시 백엔드.
- 목적/대상 사용자: 1인 구매대행 셀러, 시세차익 마케터, AI 콘텐츠 크리에이터(CLAUDE.md 2장).
- 문서 상태: CLAUDE.md(PRD v5, 50KB)와 CLAUDE_v0.md(PRD v3, 35KB) 두 버전이 공존. v5조차 코드와 어긋남 - 인증을 "sessionStorage Mock"으로 기술하지만 실제는 Supabase OAuth(lib/stores/authStore.ts), 백엔드를 "Vertex AI 프록시 + PROXY_HEADER 없으면 종료"로 기술하지만 실제는 OpenRouter 프록시가 주력(backend/server.js:48), ReactFlow를 @xyflow/react v12로 기술하지만 frontend/package.json은 reactflow(v11) 사용, AI 모델명 규칙(gemini-3.1-flash, gpt-5.5, claude-sonnet-4-6)과 코드(google/gemini-2.5-flash, openai/gpt-4o, anthropic/claude-sonnet-4-5)가 불일치.

영역	기술
프론트	React(latest), Vite, TypeScript, Tailwind CDN(index.html 내 config), react-router-dom HashRouter, zustand 4, reactflow 11, react-hook-form + zod, lucide-react
백엔드	Node 20+ ESM, Express 5, express-rate-limit, google-auth-library(선택), backend/server.js 단일 파일 219줄
DB	Supabase(PostgreSQL, Auth PKCE). 코드가 쓰는 테이블: users, plan_limits, usage_logs, contents, orders. 저장소에 있는 frontend/supabase/schema.sql은 videos/affiliate_links/blog_posts로 전혀 다른 구 스키마
인프라	로컬 npm run dev(concurrently). 배포 설정·CI·테스트 없음
AI	OpenRouter(gemini-2.5-flash / gpt-4o / claude-sonnet-4-5) 텍스트 생성. Vertex AI 프록시 코드(Google AI Studio 템플릿 잔재). ElevenLabs/Typecast/ffmpeg는 주석과 목업

- 현재 완성도 추정: 약 30%.
  - 근거: 실제로 끝까지 동작하는 흐름은 "Supabase OAuth 로그인 -> 소싱 페이지에서 OpenRouter 호출 -> contents 저장 -> 결과물/주문 목록 조회" 하나다(app/dashboard/sourcing/page.tsx:120-182). 대시보드 홈은 전부 하드코딩 목 데이터(app/dashboard/page.tsx:25-150), 자동화 캔버스는 setTimeout 시뮬레이션(components/workflow/Canvas.tsx:163-187), 워크플로우 엔진/스케줄러는 빈 함수(lib/workflow/engine.ts, scheduler.ts), 영상/TTS/커머스 파이프라인은 목업(lib/actions, lib/media, lib/commerce). 주문 생성 경로 없음.
- 핵심 기능 목록
  - 랜딩(772줄) + 소셜 로그인 모달(Google/GitHub/Kakao -> Supabase PKCE)
  - 대시보드 홈(KPI/주문/파이프라인 - 목 데이터)
  - 상품 소싱: URL + 플랫폼/채널/톤/모델 선택 -> AI 리스팅 JSON 생성, 월 한도 체크, contents 저장, 이력 조회
  - 주문 관리/결과물 보관함: orders/contents 조회 전용
  - 자동화작업: ReactFlow 캔버스 6종 노드, 노드 설정 패널, 위상 정렬 시뮬레이션
  - 설정 모달: 14개 외부 서비스 API 키를 zod 검증 후 localStorage 저장

2. 구조 분석

경로	역할	비고
package.json(루트)	npm workspaces(frontend/backend), concurrently로 동시 실행	concurrently: latest 고정 안 됨

경로	역할	비고
backend/server.js	/api/ai/generate(OpenRouter 프록시, 15분 60회 IP 제한), /api-proxy(Vertex, 선택), /health	CORS *, 인증 없음(3장 #1)
backend/.env.local, frontend/.env.local	OPENROUTER_API_KEY / VITE_SUPABASE_*	gitignore 처리됨, git 추적 없음(확인)
frontend/index.tsx, App.tsx	엔트리. vertex-ai-proxy-interceptor.js를 전역 import, HashRouter 라우팅, ProtectedRoute/PublicRoute, PKCE 콜백 처리	인터셉터는 window.fetch를 전역 패치(3장 #6)
frontend/lib/stores/*	zustand 5개 스토어(auth/user/sourcing/ui/workflow)	sourcingStore, uiStore는 어디서도 import되지 않음
frontend/lib/supabase/{client,db,usage}.ts	Supabase 클라이언트, contents/orders/users/plan_limits CRUD, 월 사용량 집계	한도 체크가 클라이언트 전용(3장 #2)
frontend/lib/ai/index.ts	백엔드 프록시 호출 래퍼 + 프롬프트 3종	구조 양호
frontend/app/dashboard/*	Next.js 식 폴더명이지만 실제로는 react-router에서 import하는 일반 컴포넌트	create/, create/detail/, detail-builder/, settings/, (auth)/login/은 라우트에서 리다이렉트 처리되어 미사용(약 2,000줄 죽은 코드)
frontend/pages/*	구세대 페이지. Landing.tsx만 사용, 나머지 7개 미사용	contexts/AuthContext.tsx, lib/supabase/auth.tsx(모의 인증 2종), components/ui.tsx(구 UI킷)도 이들만 참조
frontend/components/workflow/*	Canvas(361줄), NodePalette, 6종 노드, NodeSettingsPanel	노드 data에 apiKey 평문 보관
frontend/components/settings/SettingsModal.tsx, app/dashboard/settings/page.tsx	같은 zod 스키마·폼이 두 벌(510줄 + 441줄)	후자는 미사용
frontend/supabase/schema.sql	구 스키마(videos 등)	현행 테이블 정의 없음(3장 #4)
frontend/dist/	빌드 산출물	gitignore 처리됨, 로컬 잔재
main.html	React UMD + Babel standalone으로 src/*.jsx를 로드하는 디자인 시안 HTML	src/ 폴더가 없어 열어도 동작 안 함. 정리 대상
structer.zip, structer_extracted/structure/*.html	"대시보드 구조 설계 v2" 정적 시안 2개	zip + 추출본이 모두 git 추적됨. zip 삭제, HTML은 docs/design/으로 이동 권장
aa-extracted/design_handoff_dashboard/	대시보드 디자인 핸드오프(JSX)	gitignore 처리됨. 참고용 보관 가능하나 저장소 외부가 적절
CLAUDE_v0.md	구 PRD	v5와 중복, 삭제 또는 docs/archive/ 이동

데이터 흐름 요약: 브라우저 SPA가 Supabase에 anon 키로 직접 읽고 쓰며(contents, usage_logs insert 포함), AI 호출만 Vite 프록시를 거쳐 Express(localhost:5000)로 보내고 Express가 OpenRouter 키를 붙여 중계한다. 플랜 한도(checkLimit)는 브라우저에서 plan_limits와 usage_logs를 읽어 판단하고 통과 시 AI를 호출한 뒤 사용량을 브라우저가 기록한다. 즉 과금·한도의 모든 판단 지점이 클라이언트에 있고 서버는 "키 보관 + IP 레이트리밋"만 담당한다. 대시보드 홈/자동화는 데이터 소스가 없는 순수 UI다.

3. 발견된 문제점

#	심각도	분류	위치	내용
1	높음	보안	backend/server.js:19-25,48-53	/api/ai/generate가 CORS * + 인증 없음. IP당 15분 60회 제한만 있어 공개 배포 시 누구나 OpenRouter 크레딧 소진 가능. Supabase JWT 검증 없음
2	높음	보안	frontend/lib/supabase/usage.ts:49-69, app/dashboard/sourcing/page.tsx:125-134,173	플랜 한도 체크와 usage_logs 기록이 전부 클라이언트. 체크 실패 시에도 진행(catch { / 진행 / }). 콘솔에서 /api/ai/generate를 직접 호출하면 한도 무관
3	높음	보안	components/settings/SettingsModal.tsx:12-27,136, lib/actions/index.ts:27-29, lib/media/index.ts:19-23	OpenAI/Anthropic/ElevenLabs/쿠팡/아마존 등 14개 서비스의 API 키·시크릿을 localStorage('nomad-ai-settings')에 평문 저장하고 브라우저에서 직접 외부 API 호출을 전제로 설계. XSS 1건으로 전부 유출. V.2 CLAUDE.md 0장 "localStorage 금지" 규칙과 정면 충돌
4	높음	설정	frontend/supabase/schema.sql vs lib/supabase/db.ts:56-127, usage.ts:16	저장소 스키마는 videos/affiliate_links/blog_posts인데 코드는 users/plan_limits/usage_logs/contents/orders 사용. 현행 테이블 DDL·RLS가 저장소에 없어 재현 불가. usage_logs INSERT RLS가 느슨하면 사용자가 타인 user_id로 기록 가능
5	중간	설정	frontend/package.json:12-29	의존성 12개가 "latest". React/Vite/TypeScript 메이저 업데이트 시 빌드가 비결정적으로 깨짐. 루트 concurrently도 동일
6	중간	구조	frontend/index.tsx:1, frontend/vertex-ai-proxy-interceptor.js:10-11, index.html:16-22	Google AI Studio 템플릿 잔재: 전역 window.fetch/WebSocket 패치, window.process.env.API_KEY를 localStorage('ai-studio-api-keys')에서 주입, vite.config.ts:9-10 더미 API_KEY define. 현재 Vertex 경로 미사용
7	중간	품질	frontend/pages/*(Landing 제외 7개), contexts/AuthContext.tsx, lib/supabase/auth.tsx, components/ui.tsx, app/(auth)/login/, app/dashboard/create/**, detail-builder/, settings/page.tsx, components/workflow/Sidebar.tsx, components/forms/CreateForm.tsx, lib/actions, lib/media, lib/commerce, common/PlanGate.tsx, stores/sourcingStore.ts, stores/uiStore.ts	import 그래프 기준 미사용 파일 20여 개, 약 4,500줄(전체 11,187줄의 40%). 인증 구현이 3벌(AuthContext, supabase/auth.tsx, authStore)
8	중간	품질	frontend/app/dashboard/settings/page.tsx:40-224, components/settings/SettingsModal.tsx:42-47, components/workflow/Canvas.tsx:88,129	as any 37건. 특히 react-hook-form 경로를 as any로 우회해 zod 스키마 타입 안전성이 무력화
9	중간	버그	frontend/package.json:21 + index.html:13 + CLAUDE.md 7장	reactflow(v11) npm 패키지 사용하면서 CSS는 esm.sh/reactflow@11.10.4 CDN에서 로드, 문서는 @xyflow/react v12로 명시. 버전 불일치 시 스타일 깨짐
10	중간	구조	frontend/app/dashboard/page.tsx:25-150, components/workflow/Canvas.tsx:163-187, lib/workflow/engine.ts, scheduler.ts	대시보드 KPI/주문/채널 전부 하드코딩, 자동화 실행은 setTimeout 시뮬레이션, 엔진·스케줄러 빈 함수. "완료"로 표기된 문서와 괴리
11	중간	성능	frontend/lib/stores/userStore.ts:24-56, lib/supabase/usage.ts:49-68	로그인마다 users -> plan_limits -> usage_logs x3 순차 쿼리(최소 5회 라운드트립). 소싱 1회마다 다시 users+plan_limits+usage_logs 3회 조회
12	낮음	품질	frontend/app/dashboard/*(730/748/629/549/458/447/441/431줄), pages/Landing.tsx(772줄)	페이지 컴포넌트 8개가 400~770줄. 상수·목 데이터·서브 컴포넌트가 한 파일에 혼재

#	심각도	분류	위치	내용
13	낮음	품질	frontend/lib/actions/index.ts, lib/commerce/index.ts, backend/server.js:75,96	console.log 36건(이모지 포함). 운영 로깅 체계 없음
14	낮음	설정	backend/server.js:7 + backend/package.json:10-11	import 'dotenv/config'와 node --env-file=.env.local 이중 로딩. dotenv는 .env만 읽어 실효 없음
15	낮음	설정	structer.zip, structer_extracted/, main.html, CLAUDE_v0.md, aa-extracted/	디자인 시안·구 PRD·zip 추출 잔재가 코드와 같은 레벨에 혼재. main.html은 참조하는 src/*.jsx가 없어 깨진 파일
16	낮음	버그	frontend/lib/ai/index.ts:131, backend/server.js:68-70	환율을 모델이 "추정"한 값으로 사용(기본 1380 하드코딩). 마진·판매가 계산 근거가 LLM 환각에 의존. temperature 0.7로 JSON 모드 수치 생성

심각도 높음 상세

#1 공개 AI 프록시. backend/server.js:48의 /api/ai/generate는 요청자 확인 없이 OPENROUTER_API_KEY로 중계한다. 모델 화이트리스트(OR_MODEL_MAP)와 max_tokens: 4096은 좋지만, CORS가 *(:20)이므로 어떤 사이트에서든 호출 가능하고 IP 기반 레이트리밋은 분산 요청에 무력하다. 프론트가 이미 Supabase 세션을 가지고 있으므로 Authorization: Bearer <access_token>을 보내고 서버가 supabase.auth.getUser(token)으로 검증한 뒤 사용자 단위 한도를 적용해야 한다.

#2 클라이언트 측 한도/과금 판단. checkLimit(usage.ts:49)은 브라우저가 users.plan과 plan_limits를 읽어 비교하고, 통과 후 logUsage(:11)로 브라우저가 사용량을 기록한다. sourcing/page.tsx:133은 한도 조회 실패 시 그대로 진행하고 :174는 기록 실패를 무시한다. 사용자가 logUsage를 호출하지 않거나 plan_limits를 캐시 조작하면 무제한 사용이 되므로, 한도 체크와 사용량 기록은 #1의 백엔드(또는 Supabase RPC/Edge Function)로 이동해 AI 호출과 같은 트랜잭션에서 처리해야 한다.

#3 외부 API 키 localStorage 보관. SettingsModal.tsx는 OpenAI·Anthropic·ElevenLabs·Typecast·쿠팡·아마존·알리익스프레스 키/시크릿을 받아 localStorage에 JSON으로 저장(:136)하고, lib/actions/index.ts:27-29와 lib/media/index.ts:40,60은 그 키로 브라우저에서 직접 외부 API를 호출하도록 설계되어 있다(현재는 목업). 실제 연결 시 모든 키가 XSS·확장프로그램·공용 PC에 노출된다. 키는 서버(Supabase user_api_keys + pgsodium 암호화)에 저장하고 서버가 대신 호출해야 한다. V.2 CLAUDE.md의 절대 규칙("localStorage 금지", "API 키 .env.local")과도 상충한다.

#4 스키마 부재. db.ts/usage.ts가 의존하는 5개 테이블의 DDL, RLS, plan_limits 시드 데이터가 저장소에 없다. 저장소의 schema.sql은 폐기된 영상 파이프라인용이다. 새 환경에서 프로젝트를 재현할 수 없고, 특히 usage_logs/contents에 WITH CHECK (auth.uid() = user_id) INSERT 정책이 있는지 코드로 확인할 수 없어 #2의 심각도를 키운다.

V.2 폴더와의 관계 및 v1 코드 처분 권고

Nomad-AI-Factory_V.2는 코드 없이 기획 문서 10개(PRD v3.0.0, CLAUDE.md v3.0.0, 디자인/마켓 스펙)만 있다. V.2가 확정한 스택은 **Next.js 16 App Router + Tailwind v4 + shadcn + Zustand 5 + TanStack Query + @xyflow/react + Supabase RLS + TossPayments**이며, 아키텍처는 멀티테넌트 SaaS(RLS user_id 격리, 4단계 role)다. v1은 Vite SPA + HashRouter + Tailwind CDN + reactflow v11 + Express 프록시이므로 프레임워크·라우팅·스타일링·캔버스 라이브러리·인증 경계가 전부 다르다. 결론: **v1 저장소는 폐기(아카이브)하고 V.2는 새 Next.js 프로젝트로 시작하되, 아래 자산만 이식한다.**

재사용 가치 있음(이식)	이유
frontend/lib/ai/index.ts의 프롬프트 3종(리스팅 JSON 스키마, 톤/채널 가이드, 블로그/SNS)	도메인 지식이 담겨 있고 Route Handler로 그대로 옮길 수 있음
backend/server.js의 OpenRouter 호출 패턴(모델 맵, json_object, 에러 처리)	lib/llm/openrouter.ts로 변환 가능

재사용 가치 있음(이식)	이유
components/workflow/Canvas.tsx의 위상 정렬 실행 순서, 6종 노드 분류, NodeSettingsPanel UX	@xyflow/react v12 API로 재작성 필요하지만 설계는 유효
lib/stores/authStore.ts의 PKCE/HashRouter 이슈 해결 주석	Next.js에서는 불필요하지만 Supabase OAuth 이벤트 순서 지식은 참고
app/dashboard/sourcing/page.tsx의 상수(플랫폼/채널/톤 옵션)	데이터로 분리해 재사용

폐기	이유
pages/ 폴더, contexts/ 폴더, lib/supabase/auth.tsx, components/ui.tsx, app/dashboard/create, detail-builder, settings, lib/actions, lib/media, lib/commerce, CreateForm	이미 미사용인 영상 파이프라인 세대 코드
Tailwind CDN config, index.html 커스텀 CSS, HashRouter 라우팅	Next.js/Tailwind v4로 전환 시 무의미
vertex-ai-proxy-interceptor.js, /api-proxy, vite.config.ts 더미 define	Google AI Studio 템플릿 잔재
SettingsModal localStorage 키 저장 방식	V.2 보안 규칙 위반
main.html, structer.zip, structer_extracted/, CLAUDE_v0.md	시안·구 문서. 필요한 HTML만 docs/design/으로

v1을 계속 운영할 계획이 없다면 아래 5장의 P0 보안 작업(1~3번)도 V.2 신규 구현에서 처음부터 반영하는 것이 비용상 유리하다. v1을 당분간 데모로 유지해야 할 경우에만 1~3번을 v1에 적용한다.

4. 효율성 개선 제안

구조/아키텍처

- (V.2 전제) Next.js Route Handler로 AI 호출·한도 체크·사용량 기록을 한 지점에서 처리. Express 백엔드 제거.
- (v1 유지 시) backend/server.js에 Supabase JWT 검증 미들웨어 추가, CORS origin을 프론트 도메인으로 제한, 한도 체크/사용량 기록을 서버로 이동.
- 외부 서비스 API 키는 사용자별 서버 저장(암호화)으로 전환, 브라우저에서 외부 API 직접 호출 금지.
- 미사용 파일 20여 개 일괄 삭제 후 tsc --noEmit으로 무결성 확인. 인증 구현을 authStore 하나로.

성능

- userStore.loadProfile의 순차 5쿼리를 Supabase RPC 1회(get_user_dashboard)로 통합하거나 plan_limits를 정적 상수로.
- 대시보드 KPI는 orders/contents 집계 뷰(materialized view 또는 RPC)로 교체해 목 데이터 제거.
- Landing.tsx 772줄과 대시보드 페이지들은 코드 스플리팅(React.lazy) 또는 Next.js 라우트 단위 분리.

개발 생산성 (빌드, 타입, 테스트, 중복 제거)

- package.json 전 의존성 버전 고정(^ 범위) 및 lockfile 커밋 확인. reactflow->@xyflow/react 단일화, CDN CSS 제거.
- as any 37건 제거: react-hook-form Path<FormValues> 타입 사용, Canvas rfInstance는 ReactFlowInstance 타입.
- 현행 스키마 DDL + RLS + plan_limits 시드를 supabase/migrations/로 추출(supabase db dump).
- CLAUDE.md를 코드 기준으로 갱신(인증=Supabase OAuth, 백엔드=OpenRouter, reactflow 버전, 모델명). CLAUDE_v0.md 제거.
- 최소 테스트: generateProductListing JSON 파싱, topoSort, checkLimit 경계값.

운영/배포

- console.log 36건을 로거로 대체하고 운영 빌드에서 제거. 백엔드는 요청 ID·모델·토큰 사용량만 구조화 로그.
- .env.example 2개(frontend/backend) 추가. dotenv 또는 --env-file 중 하나만 사용.

- 저장소 정리: structer.zip 삭제, structer_extracted/structure/*.html과 main.html은 docs/design/으로 이동(또는 삭제), aa-extracted/는 저장소 외부 보관.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	V.2 신규 Next.js 16 프로젝트 골격 설계·초기화: (auth)/(dashboard) 라우트 그룹, Supabase SSR 클라이언트, proxy.ts 라우트 보호, RLS 포함 초기 마이그레이션(users/plan_limits/usage_logs/contents/orders), 멀티테넌트 role 정책	P0	L	Fable 5.1	기획안만 있는 차기 버전의 아키텍처·데이터 모델·인증 설계
2	AI 호출 경계 재설계: Route Handler(/api/ai/*)에서 인증 -> 플랜 한도 체크 -> OpenRouter 호출 -> 사용량 기록을 단일 트랜잭션(Supabase RPC)으로 처리. v1 lib/ai 프롬프트 이식	P0	M	Fable 5.1	과금·한도 로직, 실수 시 비용 누출
3	사용자 외부 API 키 서버측 암호화 저장(user_api_keys + pgsodium) 및 서버 대리 호출 구조 설계. localStorage 저장 제거	P0	M	Fable 5.1	보안 민감 데이터 저장 설계
4	(v1 유지 시 한정) backend/server.js에 Supabase JWT 검증 + CORS origin 제한 + 사용자별 한도 체크 추가	P0	S	Fable 5.1	인증·보안 수정, 파일은 1개지만 실수 비용 큼
5	v1 저장소 정리: 미사용 파일 20여 개 삭제, 인증 구현 단일화(authStore), vertex-ai-proxy-interceptor.js/api-proxy/더미 define 제거, structer.zip·main.html·CLAUDE_v0.md 정리, 시안 HTML docs/design/ 이동	P1	M	Sonnet 5.5	반복적 삭제·이동 작업, tsc로 검증 가능
6	의존성 버전 고정, reactflow->@xyflow/react v12 마이그레이션(Canvas/노드 6종/NodeSettingsPanel)	P1	M	Opus 5.5	명확한 API 마이그레이션, 3~9개 파일
7	as any 37건 제거(react-hook-form Path 타입, ReactFlowInstance), console.log 36건 로거 대체	P1	S	Sonnet 5.5	린트/타입 정리
8	대시보드 홈 목 데이터를 orders/contents 집계 RPC로 교체, userStore 5쿼리를 RPC 1회로 통합	P1	M	Opus 5.5	데이터 페칭 정리, API 연동
9	자동화 캔버스 실행 엔진 실제 구현: 노드 타입별 실행기(소싱->AI->블로그->SNS), 위상 정렬 실행, 실행 로그 테이블, 미연결 노드 실행 차단(V.2 규칙)	P1	L	Fable 5.1	다중 모듈에 걸친 엔진 설계
10	소싱 결과의 환율을 LLM 추정값 대신 실시간 환율 API(한국수출입은행) 값으로 계산, 마진 공식(CLAUDE.md 11장)을 서버 유틸로 구현 + 단위 테스트	P1	S	Opus 5.5	명확한 계산 로직 + 테스트 설계
11	대형 페이지 분해: sourcing/page.tsx(748줄), Landing.tsx(772줄) 등을 섹션 컴포넌트·상수 파일로 분리	P2	M	Sonnet 5.5	반복적 컴포넌트 분리
12	CLAUDE.md를 실제 코드 기준으로 전면 갱신(인증·백엔드·버전·모델명), .env.example 2종, README 재작성	P2	S	Sonnet 5.5	문서화·설정 정리
13	주문 생성/상태 변경 CRUD 및 결과물 삭제·재생성 기능(현재 조회 전용)	P2	M	Opus 5.5	정의 명확한 중간 규모 기능

6. 종합 평가

- 강점

1. AI 키를 백엔드에만 두고 모델 화이트리스트·레이트리밋·json_object 모드를 적용한 프록시 패턴은 방향이 옳다.
2. Supabase PKCE + HashRouter 조합의 이벤트 순서 문제를 해결한 `authStore`/`OAuthCallbackHandler`는 실제 디버깅을 거친 코드로, 로그인 흐름이 동작한다.
3. 소싱 페이지의 프롬프트(채널·톤 가이드, JSON 스키마 강제, 코드펜스 제거 파싱)와 자동화 캔버스 UX(위상 정렬, 노드 설정 패널)는 V.2에 이식할 가치가 있다.

- 약점
  1. 한도·과금·사용량 기록이 전부 클라이언트에 있고 AI 프록시에 인증이 없어 공개 배포 시 비용 통제가 불가능하다.
  2. 전체 코드의 약 40%가 미사용(세대가 다른 영상 파이프라인 잔재), 인증 구현 3벌, 설정 폼 2벌 등 중복이 심하고 문서(PRD v5)가 코드와 불일치한다.
  3. V.2가 확정한 스택(Next.js 16, Tailwind v4, @xyflow v12, 멀티테넌트 RLS)과 v1 스택이 근본적으로 달라 v1 위에 V.2를 쌓을 수 없다.
- 모델별 비율(작업 13개 기준): Fable 5.1 38% (5건) / Opus 5.5 31% (4건) / Sonnet 5.5 31% (4건)
- 권장 진행 순서
  1. 1단계: 처분 결정 - v1 폐기 + V.2 신규 시작이면 작업 1->2->3 순으로 골격을 세운다. v1을 데모로 유지해야 하면 작업 4를 먼저 적용해 비용 누출을 막는다.
  2. 2단계: 작업 5~10 - v1에서 이식할 자산(프롬프트, 캔버스 설계, 상수)을 V.2로 옮기면서 저장소 정리·버전 고정·타입 정리·대시보드 실데이터·엔진 구현을 병행한다.
  3. 3단계: 작업 11~13 - 대형 컴포넌트 분해, 문서 갱신, 주문/결과물 CRUD 보강.
- 주 담당 모델: **Fable 5.1**. 이 프로젝트의 다음 단계는 "기존 코드 손질"이 아니라 "V.2 기획안을 Next.js 멀티테넌트 SaaS로 설계해 초기 골격을 세우는 일"이고, 그 핵심이 인증 경계·플랜 한도 트랜잭션·사용자 키 암호화 저장·자동화 엔진 같은 아키텍처 결정이다. 골격이 잡힌 뒤 reactflow 마이그레이션·RPC 연동·CRUD는 Opus 5.5, 대량 삭제·타입 정리·문서 갱신은 Sonnet 5.5로 넘기는 분담이 적절하다.